



Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 — Deep Learning y Sistemas Inteligentes

Hoja de trabajo #2

Transformers y mecanismos de atención

Nicolás Concuá
Guatemala, octubre de 2026

1. Investigación: de las RNN a la atención

- **Cuello de botella del seq2seq.** El encoder RNN resume toda la oración en un único vector de tamaño fijo (su último estado) y el decoder solo ve ese vector [4]. Su capacidad no crece con la entrada: con oraciones largas hay que comprimir más en el mismo espacio y lo del inicio se diluye tras muchos pasos de recurrencia. **Bahdanau [2]** conserva todos los estados h_j del encoder (bidireccional) y en cada paso calcula $e_{ij} = v^T \tanh(W s_{i-1} + U h_j)$, $\alpha_{ij} = \text{softmax}(e_{ij})$ y el contexto $c_i = \sum \alpha_{ij} h_j$: cada palabra de salida mira directamente las palabras de entrada relevantes (alineamiento suave).
- **Aditiva vs. multiplicativa.** Bahdanau compara consulta y llave con una MLP pequeña; Luong [3] con el producto punto $s \cdot h$ (o $s^T W h$). La de producto punto es más eficiente en hardware moderno: todos los puntajes salen de una sola multiplicación de matrices QK^T (GEMM en tensor cores), mientras la aditiva materializa un tensor $n \times m \times d_a$ con $\tanh$ elemento a elemento. Con d_k grande los productos crecen, de ahí el factor $\sqrt{d_k}$ [1].
- **Q, K, V como diccionario suave.** Un diccionario devuelve el valor de la llave idéntica a la consulta; la atención compara la consulta con todas las llaves, convierte las similitudes en pesos que suman 1 y devuelve el promedio ponderado de todos los valores: una búsqueda difusa y diferenciable. En **self-attention** $Q = XW_Q$, $K = XW_K$, $V = XW_V$ salen de la misma secuencia; en **cross-attention** Q viene del decoder y K, V de la salida del encoder.
- **Qué elimina el Transformer.** El cómputo secuencial ($O(n)$ pasos dependientes $\rightarrow O(1)$ por capa, todo en paralelo), el camino $O(k)$ entre tokens a distancia k ($\rightarrow O(1)$: todo par está conectado en una capa) y la cadena de Jacobianos que desvanece el gradiente [5] (el gradiente cruza una atención y conexiones residuales). A cambio, el costo es $O(n^2)$ y hay que inyectar el orden con codificación posicional.

2. Investigación: la arquitectura Transformer

- **Por qué $\sqrt{d_k}$.** Si q_i, k_i son independientes con media 0 y varianza 1, $q \cdot k = \sum q_i k_i$ suma d_k términos de varianza $E[q_i^2]E[k_i^2] = 1$, así que **$\text{Var}(q \cdot k) = d_k$** . Con $d_k = 64$ los puntajes tienen desviación 8, la softmax se satura (casi one-hot) y su Jacobiano $\text{diag}(p) - pp^T \rightarrow 0$: los gradientes se desvanecen. Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1 (verificado en la Figura 1).
- **Multi-head.** h cabezas de d_{model}/h , cada una con sus proyecciones, permiten atender en paralelo a relaciones distintas (posición, sintaxis, correferencia) en subespacios distintos; una sola cabeza las mezclaría en un promedio. **Los parámetros no cambian:** las h matrices $d_{\text{model}} \times d_k$ suman $d_{\text{model}} \times d_{\text{model}}$; total $4 \cdot d_{\text{model}}^2 + 4 \cdot d_{\text{model}}$ (16,640 con $d = 64$ y $h = 1, 4, 8$ o 16 en el notebook).
- **Codificación posicional.** La self-attention es equivariante a permutaciones (reordenar la entrada reordena la salida; verificado en 6.1), así que no distingue el orden. La sinusoidal suma $PE(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})$, $PE(\text{pos}, 2i+1) = \cos(\cdot)$: sin parámetros, $PE(\text{pos}+k)$ es lineal en $PE(\text{pos})$. Los embeddings aprendidos (BERT, GPT-2) rinden igual pero no pasan de la longitud máxima (512/1024). **RoPE** [7] rota pares de componentes de q y k según la posición en cada capa, así $q_m \cdot k_n$ depende solo de $m - n$ (relativa); es el estándar en LLaMA y se extiende reescalando frecuencias.
- **Bloque completo.** Residual $x + F(x)$ (camino directo para el gradiente), LayerNorm por token (estabiliza la escala) y FFN por posición $W_2 \cdot \text{ReLU}(W_1 x)$ con $d_{\text{ff}} = 4d$ (mezcla dentro del token; la atención mezcla entre tokens). **Post-LN** (original): $\text{LN}(x + F(x))$; gradientes grandes cerca de la salida al inicio, requiere warm-up y es inestable en profundidad [6]. **Pre-LN:** $x + F(\text{LN}(x)) + \text{LN}$ final; residual limpio, estable sin warm-up, usado por GPT-2/3 y LLaMA. Mismos parámetros (3,152,384 en una capa $d = 512$).
- **Complejidad.** QK^T cuesta $O(n^2 d)$ en tiempo y la matriz ocupa $O(n^2)$ por cabeza y capa. **FlashAttention** [8]: exacta, por bloques en SRAM sin materializar $n \times n$ (memoria $O(n)$), pero los FLOPs siguen $O(n^2)$ y necesita kernels específicos. **Ventanas/dispersa** (Longformer [9]): $O(n \cdot w)$,

pierde la conexión directa entre tokens lejanos. **Lineal** [10]: $\phi(Q)(\phi(K)^T V)$ en $O(n)$, aproximada y menos nítida que la softmax.

3. Investigación: atención en PyTorch y Hugging Face [11][12]

Tabla 1. Funciones y clases para implementar y analizar atención.

Función / clase	Propósito y parámetros relevantes
F.scaled_dot_product_attention	softmax($QK^T \cdot \text{scale} + \text{máscara}$)V en un kernel (Flash / memory-efficient / math); solo devuelve la salida. attn_mask booleana (True = puede atender) o flotante aditiva; dropout_p (0 en evaluación); is_causal (triangular inferior); scale (por defecto $1/\sqrt{d_k}$).
nn.MultiheadAttention	Proyecciones Q, K, V (in_proj_weight), h cabezas y out_proj. embed_dim divisible entre num_heads; batch_first (B, L, E); kdim/vdim para llaves/valores de otra dimensión (cross-attention, crea q/k/v_proj_weight separados).
forward(...)	key_padding_mask (B, S): True = padding ignorado. attn_mask (L, S): True = prohibido (al revés que F.sdpa) o aditiva. need_weights: devolver pesos. average_attn_weights: True $\rightarrow$ (B, L, S) promediado; False $\rightarrow$ (B, h, L, S).
nn.TransformerEncoderLayer	Self-attention + FFN con residual, dropout y LayerNorm. d_model, nhead, dim_feedforward (2048), dropout (0.1), norm_first (False = Post-LN, True = Pre-LN).
generate_square_subsequent_mask(n)	Máscara causal flotante $n \times n$: 0 en y bajo la diagonal, -inf encima (se suma a los puntajes).
HF output_attentions=True	outputs.attentions: tupla de un tensor por capa, cada uno (batch, cabezas, n, n), post-softmax. Requiere attn_implementation="eager" (sdpa/flash no materializan los pesos).

4. Escalamiento y costo computacional

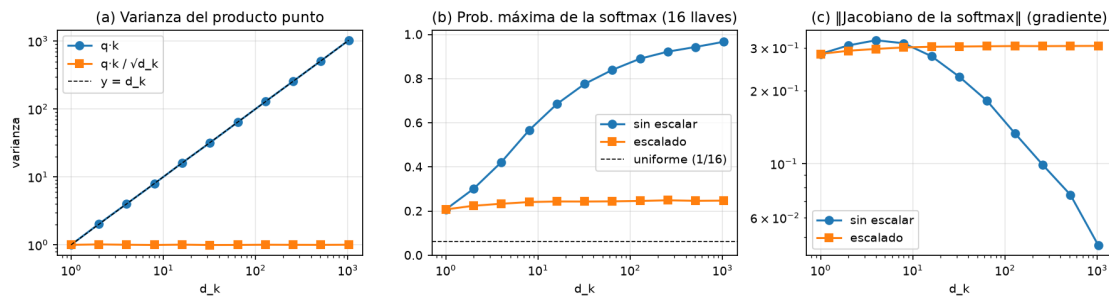


Figura 1. $q, k \sim N(0, I)$, 4,000 muestras y 16 llaves: (a) varianza de $q \cdot k$, (b) probabilidad máxima de la softmax, (c) norma del Jacobiano de la softmax.

La varianza de $q \cdot k$ sigue $y = d_k$ (63.8 con $d_k = 64$; 1,027 con 1,024) y escalada queda en 1.00. Sin escalar, con $d_k = 64$ la softmax pone **84 %** de la masa en una llave (97 % con 1,024) y la norma del Jacobiano cae de 0.30 a 0.18 (0.05): menos gradiente hacia Q y K. Con escalamiento ambas curvas quedan planas (24 %, 0.30).

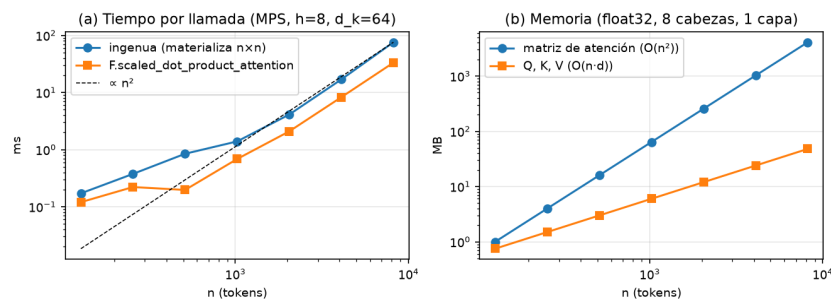


Figura 2. Atención con batch 1, 8 cabezas y $d_k = 64$ en GPU (MPS, Apple M4 Pro): (a) tiempo vs. n , (b) memoria de la matriz $n \times n$ vs. Q, K, V.

Entre $n = 2,048$ y 8,192 la pendiente log-log del tiempo es **2.09 (ingenua) y 2.01 (F.sdpa)**: duplicar n cuadruplica el costo ($n = 8,192$: 74 ms y 33 ms). El kernel fusionado es $\sim 2\times$ más rápido pero sigue siendo cuadrático. La matriz de atención de una sola capa ocupa **4 GB con $n = 8,192$** frente a 48 MB de Q, K y V.

5. Verificaciones de la implementación

Se implementaron atención(q, k, v, máscara, causal, escala) y MultiHeadPropia (proyecciones, separación en h cabezas, concatenación y W_O) y se compararon con PyTorch copiando los pesos: **13 de 13 verificaciones correctas** (Tabla 2).

Tabla 2. Verificaciones contra PyTorch (error absoluto máximo).

Verificación	Error	Verificación	Error
Atención vs F.sdpa sin máscara	4.8e-7	nn.MHA: average_attn_weights = promedio de cabezas	3.0e-8
Atención vs F.sdpa is_causal=True	3.6e-7	key_padding_mask (True = ignorar)	1.3e-7
Atención vs F.sdpa attn_mask booleana	3.6e-7	Peso sobre posiciones de padding	0
Atención vs F.sdpa scale = 0.5	4.8e-7	attn_mask causal en nn.MHA (True = prohibido)	2.1e-7
MHA propia vs nn.MultiheadAttention (salida)	1.2e-7	Encoder + generate_square_subsequent_mask: salidas 0..4 no cambian al alterar el token 5	0
MHA propia vs nn.MHA (pesos por cabeza)	8.9e-8	Filas de la softmax suman 1	1.2e-7
		Equivariancia a permutaciones (sin posiciones)	8.9e-8

6. Atención en BERT y GPT-2

bert-base-uncased y gpt2 (12 × 12 cabezas), output_attentions=True. En las 144 cabezas se midió la atención media al token anterior, siguiente, a sí mismo, [CLS], [SEP], puntuación y otra aparición de la misma palabra, y se graficó la cabeza más representativa de cada patrón (Figura 3).

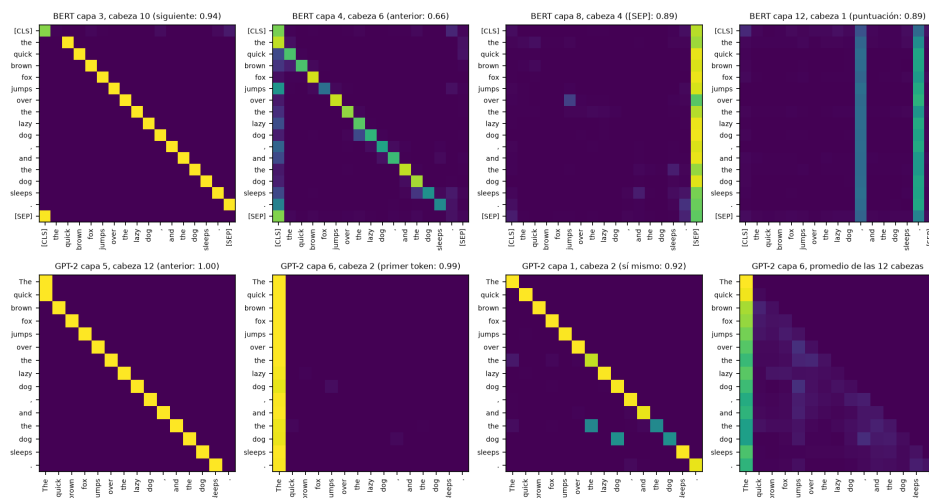


Figura 3. Mapas de atención (fila = consulta, columna = llave) para «The quick brown fox jumps over the lazy dog, and the dog sleeps.»; entre paréntesis, la fracción media de atención al patrón.

- **Patrones.** BERT tiene cabezas casi puras: 3-10 atiende al siguiente (0.94), 4-6 al anterior (0.66), 2-7 a [CLS] (0.88), 8-4 a [SEP] (0.89), 12-1 a la puntuación (0.89) y 5-4 a la otra aparición de «the»/«dog» (0.30), frente a promedios de 0.04–0.17 en las 144 cabezas. La atención a [SEP] (0.31 en promedio) es para Clark et al. [16] una operación nula. GPT-2: 5-12 atiende al anterior (1.00), 1-2 a sí misma y a la aparición previa de la palabra; el promedio de la capa 6 muestra que casi todo va al primer token.

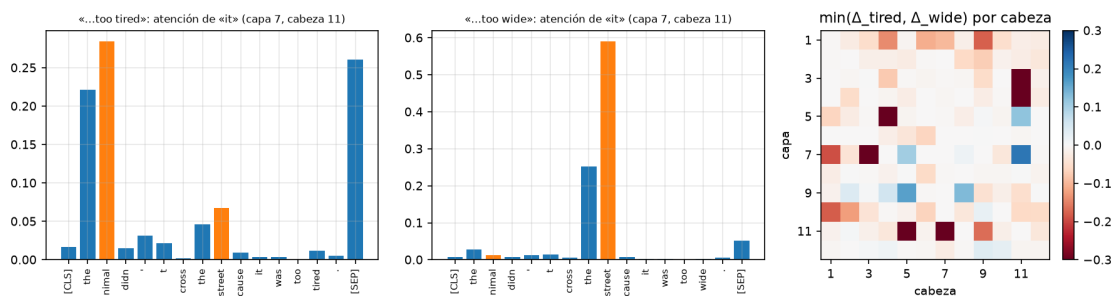


Figura 4. Atención de «it» en la cabeza 7-11 de BERT con «...too tired» y «...too wide», y $\min(\Delta_{\text{tired}}, \Delta_{\text{wide}})$ en las 144 cabezas (azul = cambio esperado).

- **Correferencia de «it».** Con $\Delta_{\text{tired}} = a(\text{it} \rightarrow \text{animal}) - a(\text{it} \rightarrow \text{street})$ en «...tired» y $\Delta_{\text{wide}} = a(\text{it} \rightarrow \text{street}) - a(\text{it} \rightarrow \text{animal})$ en «...wide», **22 de 144 cabezas** cambian en la dirección correcta y solo 3 al revés. La mejor es la **capa 7, cabeza 11**: $\text{it} \rightarrow \text{animal}$ 0.28 vs $\text{it} \rightarrow \text{street}$ 0.07 con tired; $\text{it} \rightarrow \text{street}$ 0.59 vs $\text{it} \rightarrow \text{animal}$ 0.01 con wide. Están en capas 5–9: el adjetivo viene después de «it» y debe propagarse antes. En GPT-2 la fila de «it» es idéntica en ambas oraciones ($\Delta = 0$): un modelo causal no puede resolverlo en esa posición.

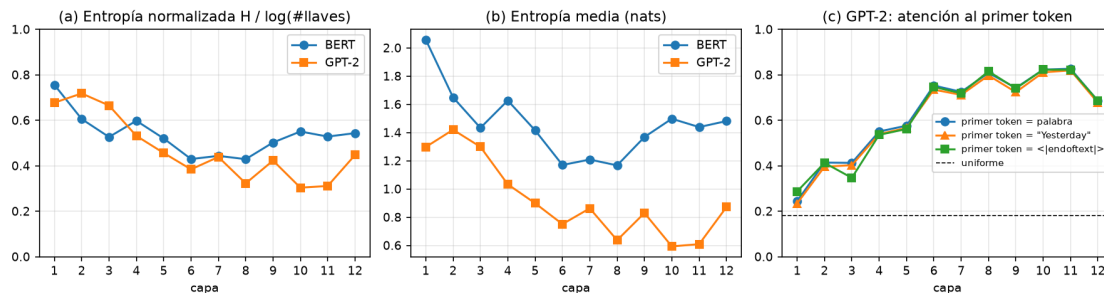


Figura 5. (a, b) Entropía de la atención por capa (normalizada por log del número de llaves visibles y en nats) en 20 oraciones; (c) fracción de atención de GPT-2 al primer token.

- **GPT-2 triangular y attention sinks.** En 20 oraciones $\times$ 144 cabezas el máximo peso encima de la diagonal es exactamente 0 y las filas suman 1 (error $2 \cdot 10^{-7}$). El primer token recibe de **24 % (capa 1) a 82 % (capas 10–11)** de la atención (uniforme: 18 %) y 103 de 144 cabezas le dan > 50 %. Es igual si el primer token es «Yesterday» o $\langle \text{endoftext} \rangle$: depende de la posición, no del contenido. Es el attention sink de Xiao et al. [13]: la softmax obliga a repartir una masa de 1 y las cabezas sin nada que atender la descargan en el token que todos ven.
- **Entropía.** Las primeras capas son las más difusas en ambos (0.76 BERT, ~ 0.7 GPT-2). GPT-2 se concentra con la profundidad (0.30 en la capa 10), en buena parte sobre el sink; BERT baja hasta las capas 6–8 (0.43, dominadas por [SEP]) y vuelve a ser más difuso al final (0.50–0.55).
- **Ablación de cabezas (BERT MLM).** Apagar una cabeza (poniendo en cero su salida antes de W_O) cambia la pérdida < 1 % en 124 de 144 casos (56 la bajan). Podando acumulativamente las menos importantes, evaluado en 20 oraciones nuevas, **apagar 42 % de las cabezas sube la pérdida solo de 1.78 a 1.88**; se dispara desde ~ 75 %.
- **Atención vs. gradiente $\times$ entrada (GPT-2).** Al predecir la última palabra de 19 oraciones, la correlación de Spearman entre la atención de la última capa y la atribución gradiente $\times$ entrada es $\rho = 0.17$ (0.06 promediando todas las capas); el primer token recibe 63 % de la atención y solo 10 % de la atribución.

7. Discusión y conclusiones

- **¿Por qué la atención permitió modelos más grandes?** Paralelismo (una capa son unas pocas multiplicaciones de matrices sobre todos los tokens: 8,192 tokens en 33 ms), camino $O(1)$ entre cualquier par de tokens, y un costo cuadrático que no domina en longitudes típicas: por capa la atención cuesta $\sim 4n^2d$ FLOPs y las proyecciones + FFN $\sim 24nd^2$, así que solo domina con $n > 6d$ ($\approx 4,600$ tokens con $d = 768$).
- **¿Se especializan las cabezas? ¿Son todas útiles?** Sí se especializan: patrones posicionales, de puntuación, de tokens especiales y de correferencia con valores de 0.66–1.00 frente a promedios < 0.2 . Pero no todas parecen útiles: muchas actúan como operación nula sobre [SEP] o el sink, y se puede quitar ~ 40 % de las cabezas con un aumento de ~ 6 % en la pérdida, como en Michel et al. [17] y Voita et al. [18].
- **¿La atención explica las decisiones?** Solo parcialmente. Como Jain y Wallace [14], encontramos poca correlación con la importancia por gradiente ($\rho = 0.17$) y el token más atendido (el sink) casi

no influye; el peso ignora la norma de los valores, la FFN, los residuales y las otras 143 cabezas. Wiegrefe y Pinter [15] matizan que la atención no es la explicación fiel pero sí puede ser una explicación plausible: la cabeza 7-11 de «it» es plausible, pero es 1 de 22 que cambian (y 3 cambian al revés). Sirve para formular hipótesis, no para afirmar causalidad.

- **¿Cuándo una RNN o una CNN?** Streaming con memoria constante por paso (estado fijo vs. caché KV que crece), dispositivos con poca memoria o latencia estricta, datos escasos donde ayuda el sesgo inductivo (localidad de la CNN, orden de la RNN) y secuencias muy largas con dependencias locales (audio crudo, series de alta frecuencia), donde son lineales en n .
- **Documentos de 100,000 tokens.** Escalando lo medido ($\times 149$ respecto a $n = 8,192$), la matriz de una capa con 8 cabezas ocuparía ~ 600 GB y tardaría ~ 11 s; además los embeddings aprendidos no llegan tan lejos (GPT-2: 1,024). Alternativas: FlashAttention (exacta, memoria $O(n)$, tiempo $O(n^2)$), ventanas + tokens globales (Longformer/BigBird), atención lineal, RoPE con interpolación de posiciones, ring attention entre varias GPU, SSM (Mamba) o RAG/fragmentación.

Conclusiones. (1) La atención elimina el cuello de botella del seq2seq; el Transformer gana paralelismo y caminos $O(1)$ a cambio de un costo $O(n^2)$ (pendiente medida ≈ 2). (2) $\sqrt{d_k}$ evita que la softmax se sature. (3) La implementación propia coincide con PyTorch (13/13). (4) Las cabezas se especializan, pero muchas son redundantes o sumideros. (5) Los pesos de atención sirven para analizar, no son una explicación fiel.

Referencias

- [1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762.
- [2] Bahdanau, D., Cho, K. y Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- [3] Luong, M.-T., Pham, H. y Manning, C. (2015). Effective Approaches to Attention-based Neural Machine Translation. EMNLP.
- [4] Sutskever, I., Vinyals, O. y Le, Q. (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.
- [5] Bengio, Y., Simard, P. y Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. IEEE TNN.
- [6] Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture. ICML.
- [7] Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
- [8] Dao, T. et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS.
- [9] Beltagy, I., Peters, M. y Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.
- [10] Katharopoulos, A. et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML.
- [11] PyTorch Documentation: scaled_dot_product_attention, MultiheadAttention, TransformerEncoderLayer. <https://pytorch.org/docs/stable/>
- [12] Hugging Face Transformers Documentation: output_attentions. <https://huggingface.co/docs/transformers>
- [13] Xiao, G. et al. (2023). Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.
- [14] Jain, S. y Wallace, B. (2019). Attention is not Explanation. NAACL.
- [15] Wiegrefe, S. y Pinter, Y. (2019). Attention is not not Explanation. EMNLP.
- [16] Clark, K. et al. (2019). What Does BERT Look At? An Analysis of BERT's Attention. BlackboxNLP.
- [17] Michel, P., Levy, O. y Neubig, G. (2019). Are Sixteen Heads Really Better than One? NeurIPS.
- [18] Voita, E. et al. (2019). Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. ACL.

Enlace al repositorio: https://github.com/nicoCT04/Hoja_trabajo_2_DL